

Hybrid Hâ`z + CNN Intelligent Boring Bar â€” Submission Package This repository contains the Hybrid Hâ`z + CNN active vibration suppression project for a CNC boring bar, with Azure Digital Twin integration and publication-quality analysis. This README provides step-by-step reproduction instructions, folder structure, commands, and expected outputs for academic submission. ## Project Overview - Hybrid control: Hâ`z robust baseline + CNN adaptive feedforward for Modeâ€²2 - Simscape plant model used for simulation and dataset generation - Edge processing packages telemetry to Azure IoT Hub and patches Azure Digital Twin ## Folder Structure (generated in `submission_package/`) - `simulink_models/` â€” .slx Simulink model files - `matlab_scripts/` â€” MATLAB scripts (.m) including Hâ`z synthesis and helpers - `python_scripts/` â€” all Python scripts used for analysis, generation, Azure integration - `datasets/` â€” generated datasets (.csv, .npz, metadata.json, telemetry JSONL) - `output_plots/` â€” publication-quality figures (PNG/SVG/PDF) - `azure_outputs/` â€” telemetry JSONL and digital twin event JSONL - `reports/` â€” FINAL_VALIDATION_REPORT.txt, validation_report.json - `documentation/` â€” architecture PDF and other docs ## Software Requirements - Python 3.8+ with: numpy, pandas, matplotlib, scipy, joblib - MATLAB + Simulink (R2022b or newer recommended) for `boring\_bar.slx` - (Optional) Azure SDKs: `azure-identity`, `azure-digitaltwins-core`, `azure.iot.device` for live integration ## Python dependencies (install via pip) ``bash pip install numpy pandas matplotlib scipy joblib # Optional (Azure live mode): pip install azure-identity azure-digitaltwins-core azure-iot-device `` ## Reproduce all final outputs â€” exact commands Run from project root `C:\Project 7`. 1. Generate datasets ``bash python scripts/generate\_datasets.py `` 2. Publication analysis (spectrograms, per-scenario figures) ``bash python scripts/publication\_analysis.py `` 3. Open-loop vs closed-loop analysis (core control plots) ``bash python scripts/analyze\_openloop\_vs\_closedloop.py `` 4. CNN performance analysis ``bash python scripts/analyze\_cnn\_performance.py `` 5. Validation metrics and PSD/time plots ``bash python scripts/validate\_metrics.py --clean output/simscape\_export\_clean.csv --fused output/integrated\_control.csv --report output/validation\_report.json `` 6. Generate Azure-compatible telemetry and events ``bash python scripts/generate\_azure\_outputs.py `` 7. Assemble final tables and report ``bash python scripts/generate\_final\_tables.py python scripts/generate\_final\_report.py `` 8. One-line full pipeline ``bash python scripts/generate_datasets.py && python scripts/publication_analysis.py && python scripts/analyze_openloop_vs_closedloop.py && python scripts/analyze_cnn_performance.py && python scripts/validate_metrics.py --clean output/simscape_export_clean.csv --fused output/integrated_control.csv --report output/validation_report.json && python scripts/generate_azure_outputs.py && python scripts/generate_final_tables.py && python scripts/generate_final_report.py `` ## Expected outputs (selection) - `output/openloop\_vs\_closedloop.png` â€” Core time/frequency control comparison - `output/mode2\_psd\_zoom.png` â€” Modeâ€²2 (500â€”700 Hz) PSD zoom - `output/cnn\_contribution\_over\_time.png` â€” CNN correction time series - `output/control\_effort\_comparison.png` â€” Energy tradeoff plot - `output/azure\_telemetry.jsonl` â€” telemetry JSONL (edge â†’ IoT Hub) - `output/digital\_twin\_events.jsonl` â€” event packets (chatter warnings, PM) - `output/azure\_architecture\_diagram.pdf` â€” architecture diagram - `output/FINAL\_VALIDATION\_REPORT.txt` â€” final validation report - `output/final\_performance\_table.csv/.tex` â€” final performance table ## Azure Digital Twin & IoT notes - Telemetry keys: `vibrationAmplitude`, `spindleSpeed`, `cnnCorrection`, `actuatorForce`, `mode2Residual`, `predictionConfidence`. - Events: `chatter\_warning`, `saturation\_alert`, `predictive\_maintenance` â€” saved in `output/digital\_twin\_events.jsonl`. - Local buffering behaviour is simulated; the edge buffers telemetry when disconnected and flushes on reconnect. ## Troubleshooting - If plotting fails, verify `matplotlib` and `scipy` are installed. - If `main.py` --live fails, confirm ADT and IoT Hub credentials in `config.py` and environment variables. - If Simulink model cannot be opened, verify MATLAB/Simulink version compatibility. ## License & Credits Project prepared for academic evaluation. See repository for author and license details. Project 7 â€” CNN Mode-2 Feedforward ===== Quick start ----- 1. Create a Python virtual environment and activate it. Windows (PowerShell): ``powershell python -m venv .venv .\venv\Scripts\Activate.ps1 `` macOS / Linux: ``bash python -m venv .venv source .venv/bin/activate `` 2. Install dependencies: ``bash pip install -r requirements.txt `` Notes: `tensorflow` on Windows may be heavy; `tensorflow-cpu` is listed to provide a CPU-only build. Run smoke test ----- ``bash python cnn.py --smoke `` This will generate a synthetic CSV, train a small model, run a prediction, and print metrics. Train on your Simscape export ----- 1. Export from MATLAB/Simscape as CSV with columns: `timestamp`, `x\_sensor`, `u\_hinf`, `u\_cnn`. 2. Run: ``bash python cnn.py --data simscape\_export.csv `` Predict with saved artifacts ----- ``bash python cnn.py --predict --data simscape\_export.csv `` VS Code Python interpreter setup ----- - Open the workspace in VS Code. - Open the Command Palette (Ctrl+Shift+P) â†’ `Python: Select Interpreter` â†’ choose the `.venv` created above. - Install the Python extension if prompted. MATLAB / Simulink ----- - `boring\_bar\_params\_1.m` is present but requires MATLAB (Control System Toolbox / Robust Control / Simulink) to run. I can't execute MATLAB code here. Files produced ----- - `model\_artifacts/` â€” trained model, scaler, and metadata - `output/` â€” copies of the artifacts and smoke log Screenshots ----- Presentation-ready screenshots are included in the `screenshots/` folder. These are placeholder images you can replace with high-resolution captures from your environment: - `screenshots/simulink\_model.svg` - `screenshots/azure\_digital\_twin\_explorer.svg` - `screenshots/telemetry\_dryrun.svg` - `screenshots/validation\_plots.svg` - `screenshots/github\_overview.svg` Architecture Diagram ----- A system architecture diagram is provided in `docs/system\_architecture.svg`. It visualizes the flow: Simulink/Simscape â†’ Hâ`z Robust Controller â†’ CNN Optimization â†’ Hybrid Fused Controller â†’ CSV Telemetry Export â†’ Python Telemetry Pipeline â†’ Azure IoT Hub â†’ Azure Digital Twin Telemetry pipeline ----- The `main.py` + `telemetry_reader.py` implements a merged telemetry stream that combines CSV exports (Simscape+CNN+fusion) into a normalized telemetry payload and supports a dry-run mode. This is suitable for demos and integration testing before connecting to live Azure resources (see `setup.md`). If you want, I can: - Pin exact package versions into a `pyproject.toml` or `constraints.txt`. - Create a small `run.sh` / `run.ps1` helper script.